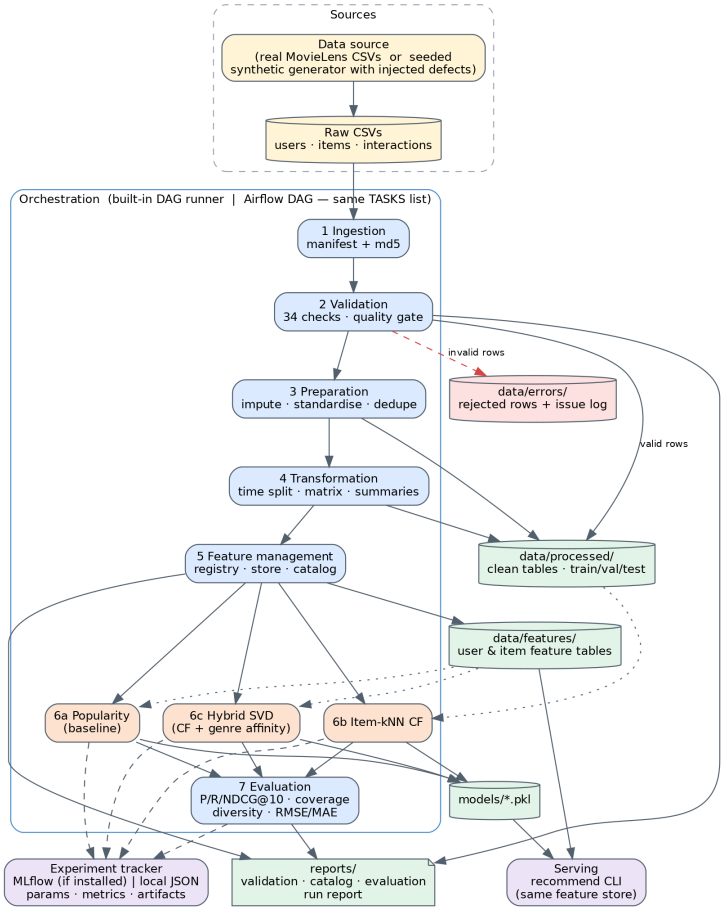
End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning – Assignment 2 · final report

1. Overview and design decisions

The pipeline recommends movies and runs as 10 dependency-ordered tasks: ingestion, validation, preparation, transformation, feature management, training of the models, evaluation, with orchestration over all of them. Every stage reads and writes files (Parquet/JSON/NPZ/pickle) under *data/*, *models/* and *reports/*, so each stage can be re-run, inspected and tested on its own. All stochastic steps (tuning, factorisation, SGD) use fixed seeds, and each stage reads only the files written by the previous one.



- **Data.** Real **MovieLens** ratings (GroupLens; 6,040 users, 3,883 movies, 1,000,209 ratings) converted to the pipeline schema by *scripts/prepare_movielens.py*; gender and an age bracket are available (age = bracket midpoint) while country is absent and imputed as UNK; signup date is the user's first rating.
- **Validation before cleaning.** 34 checks (great_expectations 1.24.1) run on the raw string data. *Errors* quarantine a record into *data/errors/* with the reason and original line number; *warnings* (e.g. a missing value) keep the record and are repaired in preparation. A quality gate (reject rate > 15%) fails the run, and downstream tasks are marked *upstream_failed*; I verified this behaviour on a copy of the project with a stricter threshold.
- **Preparation / transformation.** Imputation (median age: 0 rows), standardised gender, country and genres, and one rating per (user, item) (latest kept; 0 older duplicates dropped). A chronological split avoids leakage: train 800,164 / test 200,045 rows, with a validation slice used only for tuning. Derived datasets (user activity summary, item popularity, sparse user-item matrix) come from train only.
- **Feature management.** 12 user, 8 item and 3 user-item feature definitions live in one registry; the catalog is generated from it and checked against the materialised tables on every run. Features are published to **Feast** (user and item feature views), retrieved for training with point-in-time joins at the split date and served online from the SQLite online store; an online-vs-offline parity check guards against training/serving skew. Features are computed on train only and recency is measured against the split date. While building it I found and fixed an index-alignment bug that shifted genres between items; a unit test now guards it.

2. Models, tools and orchestration

- **Popularity** (baseline): rank by training interaction count, Bayesian rating as tie-break and rating predictor.
- **Item-kNN CF**: cosine item-item similarity on the 'liked' (rating ≥ 4) matrix, top-N neighbours, shrinkage.
- **Hybrid SVD**: PureSVD latent score on liked interactions blended with a content signal (user genre affinity from the feature store), plus a biased-SVD rating head giving RMSE/MAE. Hyper-parameters are chosen by validation NDCG@10 (explicit rank by validation RMSE) and the model is refit on all pre-test data.
- **Surprise SVD**: scikit-surprise's biased SVD on explicit ratings; ranks by predicted rating, so it is strong on RMSE/MAE but typically weaker for top-N.
- **Tools.** Python, pandas, NumPy, SciPy; Great Expectations (validation); Feast (features); Parquet storage; MLflow (tracking); scikit-surprise; Graphviz/matplotlib for figures; pytest. 8 tracked runs. Orchestration: Apache Airflow DAG generated from the shared task list (successful run screenshot included); a built-in DAG runner (parallel training tasks, retries, failure propagation, HTML run report) is also provided.

3. Results

Test protocol: newest 20% of interactions; for each of 1,122 users with history and ≥1 relevant test item (rating ≥ 4), the whole catalogue minus training items is ranked and the top 10 compared with relevant test items. RMSE/MAE cover all test ratings for models with a rating head.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity	0.1996	0.0493	0.2145	0.0353	0.6524	0.9688	0.7715
item_knn	0.2292	0.0588	0.2500	0.0592	0.6926	n/a	n/a
hybrid_svd	0.2269	0.0671	0.2497	0.1290	0.6625	0.9425	0.7459
surprise_svd	0.1320	0.0364	0.1392	0.1586	0.7094	0.9373	0.7405

Both personalised models beat the popularity baseline on NDCG@10 (item_knn 1.17× and hybrid_svd 1.16× the baseline's). item_knn and hybrid_svd are effectively tied on NDCG@10 (0.2500 vs 0.2497); a gap this small is within what a single split can resolve, so neither should be called best on ranking accuracy. hybrid_svd has the best recall@10 (0.0671) and surprise_svd the widest catalogue coverage (15.9%; popularity 3.5%). Rating error is lowest for surprise_svd (RMSE 0.937, MAE 0.740) against 0.969 for popularity; scikit-surprise SVD predicts ratings well but ranks poorly (NDCG@10 0.139), because it is trained for rating error, not top-N ranking. Genre diversity of the lists: hybrid 0.66 vs popularity 0.65 - similar, so personalisation costs little diversity. Validation selected alpha=0 for the hybrid, i.e. the genre-affinity content blend did not improve validation NDCG on this dataset, so its ranking comes from the collaborative SVD component alone. All comparisons come from one chronological split without confidence intervals, so they are indicative.

Data validation summary

Table	Rows in	Rejected	Reject rate	Rows w/ warnings
users	6040	0	0.00%	6040
items	3883	0	0.00%	251
interactions	1000209	0	0.00%	0

No record violated an error-severity rule, so nothing was quarantined; warnings (kept, repaired in preparation): users.missing_country (6,040 rows); items.unknown_genre (251 rows). The companion run on the synthetic dataset, with deliberately injected defects, quarantined 5,134 records and so exercises the rejection path (*reports/validation_report.md* in the project root). Built-in runner: 10/10 tasks succeeded in 82.03 s (*reports/figures/successful_run.png*); Airflow run: *reports/figures/airflow_successful_run.png*.

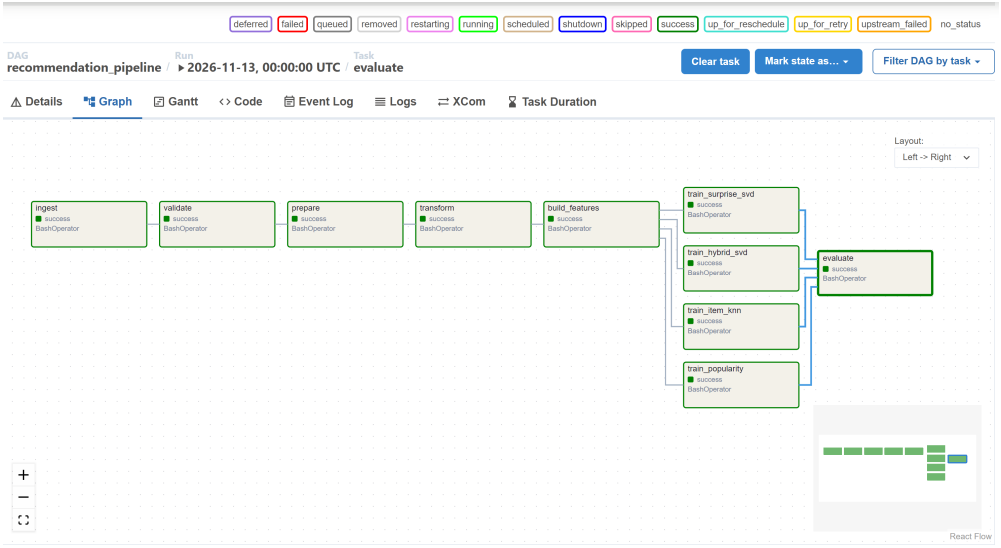


Figure: Apache Airflow graph view of the DAG run - all tasks succeeded.

4. Limitations

- **Data:** a historical movie-rating snapshot: ratings are explicit and sparse, and demographics are limited to gender and an age bracket (no country).
- **Evaluation:** one chronological split, no confidence intervals, no online test; the selected hybrid setting has k_implicit=16 on the edge of the tuning grid (a wider grid might do slightly better); cold-start users get a popularity fallback and are excluded from ranking metrics; the rating head is evaluated for 3 of 4 models (item_knn has none).
- **Scale:** item-item similarity is computed in row chunks and kept as a sparse top-N matrix (it handled 3,883 movies here), but scoring builds dense users × items blocks in memory; catalogues beyond a few hundred thousand items would need approximate neighbour search and sparse top-N retrieval.